



DopamiNah: Control de Adicción Digital

Microproyecto — Kotlin Multiplatform

Fredy, Julián

Universidad del Cauca

June 5, 2026

¿Qué es DopamiNah?

DopamiNah es un sistema multiplataforma para reducir la adicción digital mediante límites de tiempo en aplicaciones (Android) y sitios web (Web).

Características:

- Monitoreo de uso en tiempo real.
- Metas personalizables con límites diarios.
- Gamificación (rachas, insignias, niveles).
- Navegador Focus con bloqueo restrictivo.
- Sincronización con extensión de navegador.

“Toma el control de tu tiempo digital.”



Restricción en Apps Móviles (Android)

Monitoreo de uso de aplicaciones:

- Uso de `UsageStatsManager` para consultar el tiempo de uso por app.
- Metas asignadas a paquetes específicos (`com.example.app`).
- Bloqueo automático al alcanzar el límite diario mediante `App Blocker`.

Flujo de restricción:

- 1 El usuario define un límite diario para una app (ej. 30 min para Instagram).
- 2 El `DeviceUsageRepository` monitorea el tiempo acumulado.
- 3 Al superar el límite, el `App Blocker` intercepta la apertura.
- 4 Se muestra una notificación o pantalla de bloqueo.

Tecnologías: `UsageStatsManager`, `PackageManager`, `Service` en segundo plano.

Restricción en Sitios Web (Web)

Monitoreo de navegación por dominio:

- Control de tiempo por dominio (`youtube.com`, `twitter.com`).
- Integración con extensión de navegador (Chrome/Edge MV3).
- Bloqueo mediante Declarative Net Request (DNR) rules.
- Navegador Focus embebido con WebView sandbox.

Flujo de restricción:

- 1 El usuario configura un límite para un dominio (ej. 20 min para YouTube).
- 2 La extensión monitorea el tiempo activo mediante `chrome.webNavigation`.
- 3 Al agotar el tiempo, se activan reglas DNR de redirección.
- 4 El usuario ve una página de bloqueo con estadísticas del día.

Tecnologías: Extensión Manifest V3, `webNavigation`, `declarativeNetRequest`, WebView compartido.

Créditos del Proyecto

Repositorio: github.com/Crownyny/DopamiNah
Universidad del Cauca.

Desarrolladores:

- **Fredy** — Android / Shared Module
- **Julián** — Android / UI / Testing

Rama:

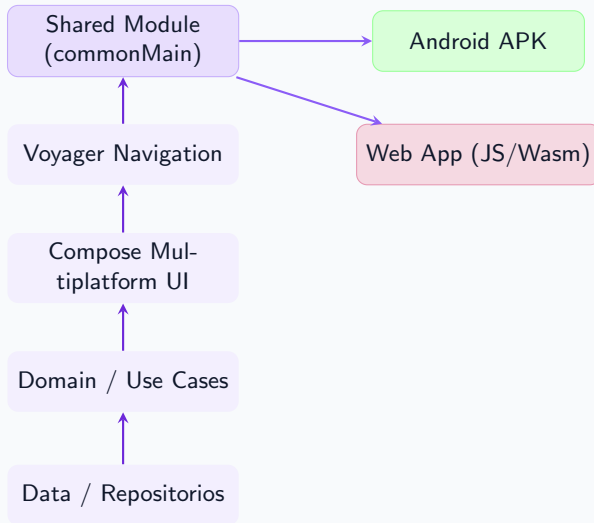
- `dev-kmp` — Desarrollo principal del proyecto KMP.

Tecnologías: Kotlin Multiplatform, Compose Multiplatform, Voyager, Koin, Multiplatform-Settings, Ktor, Coil.

Requisitos Cubiertos

- ✓ **Despliegue en 2+ plataformas** — Android y Web (JS/Wasm).
- ✓ **5+ pantallas** — Android: 6 (Onboarding, Dashboard, Stats, Goals, Achievements, Settings). Web: 4 (Goals, Navegación, Achievements, Settings).
- ✓ **Persistencia de datos** — `multiplatform-settings` (`SharedPreferences/NSUserDefaults`), próximamente `SQLDelight`.
- ✓ **Navegación en capa compartida** — Voyager + Tab Navigation gestionados en `commonMain`.
- ✓ **Repositorio GIT público** con ramas por estudiante y commits visibles.
- ✓ **Vista de créditos** con descripción de la app y nombres de los estudiantes.

Arquitectura del Proyecto



Pantallas por Plataforma

Android (5 pantallas)

- **Onboarding** — Permisos de uso (1 vez).
- **Dashboard (Inicio)** — Resumen diario y apps más usadas.
- **Stats** — Gráficos de uso por app.
- **Goals (Metas)** — Límites por aplicación.
- **Achievements (Logros)** — Insignias y rachas.
- **Settings (Ajustes)** — Tema, sincronización.

Ocultar la pestaña Navegación.

Web (4 pantallas)

- **Goals (Metas)** — Límites por dominio web.
- **Navegación** — Estadísticas de navegación por dominio.
- **Achievements (Logros)** — Insignias y rachas.
- **Settings (Ajustes)** — Tema, sincronización.

Ocultar Dashboard y Stats (no aplican en Web).

Mecanismo actual: `multiplatform-settings` (librería `russhwolf`)

- **Android:** `SharedPreferences`
- **Web:** `multiplatform-settings` (JS)

Datos persistentes:

Metas y límites de tiempo por aplicación/dominio.

Estadísticas de uso diario y horario.

Estado de gamificación (rachas, puntos, nivel, insignias).

Preferencias de tema (oscuro/claro) y suscripción premium.

Estado de sincronización con la extensión de navegador.

Próximamente: Migración a `SQLDelight` (v2.0.2) como base de datos relacional.

Navegación en Capa Compartida (KMP)

Librería: Voyager 1.1.0-beta03

- La navegación se define en `commonMain` con un `TabNavigator`.
- Cada pantalla es un `Screen` de Voyager con su propio `ViewModel`.
- El `AppState` centralizado controla el flujo (Onboarding → App).
- Las plataformas solo proveen el contenedor raíz:
 - **Android:** `setContent { App() }`
 - **Web:** `ComposeWidget()` montado en el DOM.

Ejemplo de pantallas (sealed class):

```
sealed class Screen(val route: String, val title: String) {  
    object Dashboard : Screen("dashboard", "Inicio")  
    object Stats : Screen("stats", "Stats")  
    object Goals : Screen("goals", "Metas")  
    object Achievements : Screen("achievements", "Logros")  
    object Settings : Screen("settings", "Ajustes")  
    object Onboarding : Screen("onboarding", "Permisos")  
}
```

github.com/Crownyny/DopamiNah

Rama:

- dev-kmp — Desarrollo principal del proyecto KMP.

Commits destacados:

- 4d8ade1 — Initial KMP project scaffold
- 7233ba5 — Full KMP migration plan phases 1-4 and 6
- 89ab903 — Full UI screens matching Android native branch
- c405487 — Full statistics screen and data integration
- 6e06cb7 — Goals management with real-time usage tracking
- cbf0def — Comprehensive UI updates (Settings + Achievements)
- 97fdd75 — Status bar color fixed / goals section fixed
- 39f5121 — Android: Add App Blocker

Reflexión: Multiplataforma vs Nativo

Desarrollo Nativo (Android/iOS):

- Acceso completo a APIs del sistema (UsageStatsManager, NotificationManager).
- Mejor rendimiento en UI compleja y animaciones.
- Código duplicado: lógica de negocio separada por plataforma.
- Mayor tiempo de desarrollo para mantener dos o más codebases.

Kotlin Multiplatform — Ventajas:

- Una sola base de código para lógica de negocio, navegación y UI con Compose Multiplatform.
- Reducción significativa de código duplicado entre plataformas.
- Sincronización entre plataformas más sencilla (mismos repositorios, mismos ViewModels).

Kotlin Multiplatform — Desafíos:

- Limitaciones en APIs específicas (ej. monitoreo de uso requiere `expect/actual`).
- Madurez relativa de Compose Multiplatform frente a SwiftUI / Jetpack Compose nativo.
- Curva de aprendizaje inicial para configurar correctamente los targets multiplataforma.

Conclusión:

KMP es ideal para aplicaciones con lógica compartida compleja que deben ejecutarse en múltiples plataformas. Para características que dependen fuertemente del hardware o del sistema operativo nativo, el enfoque `expect/actual` permite mantener la reutilización del código sin sacrificar la funcionalidad específica de cada plataforma.

La experiencia con DopamiNah demostró que la productividad aumenta significativamente al compartir la capa de navegación, los ViewModels y la UI en Compose Multiplatform, reduciendo el tiempo de desarrollo a aproximadamente la mitad en comparación con mantener dos proyectos nativos separados.

Dashboard y Stats

[Dashboard]

[Stats]

Goals (Metas) y Achievements (Logros)

[Metas]

[Logros]

Goals (Metas por dominio) y Navegación

[Metas Web]

[Navegación]

Achievements (Logros) y Settings (Ajustes)

[Logros]

[Ajustes]



Gracias

`github.com/Crownyny/DopamiNah`

DopamiNah — “Toma el control de tu tiempo digital”

Universidad del Cauca, 2025/2026